

International Cybersecurity and Digital Forensics Academy (ICDFA)

CIP-B101: Basic Computer Skills for Digital Forensics

Lab 1A: Number Systems, ASCII, Timestamps and Data Representation

A practical foundation for interpreting digital evidence at byte, text, time and memory levels

Lab Type	Guided individual practical with report submission
Estimated Duration	2-3 hours
Recommended Platform	Kali Linux, Ubuntu Linux, WSL, or any Linux VM with Python 3 installed
Submission Format	One PDF report containing answers, screenshots, commands used, explanations and reflection
Academic Integrity	Each student must generate their own outputs and screenshots. Copied screenshots or reports should not be accepted.

Mandatory preparation before starting

Students must first go through the course slide material titled "Understand Number Systems and Timestamps". The slide is the walkthrough for this lab. Do not proceed to the practical tasks until you understand the concepts of decimal, binary, hexadecimal, ASCII, epoch time, hashing and endianness.

1. Lab Overview

Scenario: You are a beginner digital forensics trainee. During an investigation, you receive values from disk sectors, command-line outputs, logs, hashes and memory dumps. Your task is to correctly interpret numbers, text encodings, timestamps and byte order so that evidence is not misunderstood.

1.1 Learning Outcomes

- Differentiate decimal, binary and hexadecimal number systems and explain why they matter in digital forensics.
- Convert values manually and validate conversions using Linux shell arithmetic, bc and Python.
- Interpret ASCII and UTF-8 text from hexadecimal evidence strings.
- Convert Unix epoch timestamps and understand other forensic timestamp formats such as Mac Absolute Time and WebKit/Windows-style epoch references.
- Explain why echo and echo -n can generate different hashes.
- Identify big-endian and little-endian byte ordering and explain its evidence interpretation impact.
- Prepare a structured forensic worksheet/report with commands, screenshots, results and explanations.

1.2 Tools Required

Tool	Purpose	Installation/Verification
bash terminal	Run shell commands and arithmetic tests	Available by default on Linux
python3	Validate conversions and timestamp calculations	python3 --version
bc	Base conversion through command line calculator	sudo apt install -y bc
xxd	Hex dump and ASCII view of data	sudo apt install -y xxd or vim-common
date	Unix timestamp conversion	Available by default on Linux
md5sum / sha256sum	Hash verification exercises	Available by default on Linux

1.3 Student Working Folder

```
mkdir -p ~/cip-b101-lab1a
```

```
cd ~/cip-b101-lab1a
pwd
```

Screenshot requirement

Capture screenshots showing your terminal prompt and output for each main task. Your screenshots must clearly show your name or student ID in the working folder, file names, command used and result.

2. Task A - Number Systems and Manual Conversion

This task builds your ability to interpret values written in base 10, base 2 and base 16. Forensic examiners encounter hexadecimal values in memory, disk sectors, file headers, hash outputs and metadata structures.

2.1 Quick Reference Table

Decimal	Binary	Hexadecimal
0	0	0
1	1	1
2	10	2
3	11	3
4	100	4
5	101	5
6	110	6
7	111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

2.2 Practical Conversion Exercises

1. Convert 101010_2 to decimal manually. Show the weight of each bit from right to left.
2. Convert AB_{16} to decimal manually. Remember that A = 10 and B = 11.
3. Convert 126_{10} to binary manually by division by 2.
4. Convert 2738_{10} to hexadecimal manually by division by 16.
5. Convert 101010110010_2 to hexadecimal by grouping bits in fours from the right.

2.3 Validate With Linux Shell and Python

```
# Binary to decimal using shell arithmetic
echo $((2#101010))

# Hexadecimal to decimal using shell arithmetic
echo $((16#AB2))

# Decimal to binary using bc
echo "obase=2;126" | bc

# Decimal to hexadecimal using printf
printf "%X\n" 2738

# Python validation
python3 - <<'PY'
print(int('101010', 2))
print(int('AB2', 16))
print(bin(126))
print(hex(2738))
PY
```

2.4 Evidence Table to Complete

Evidence Value	Given Base	Manual Answer	Command Used to Verify	Screenshot Ref.
101010	Binary			
AB2	Hexadecimal			
126	Decimal			
2738	Decimal			
101010110010	Binary			

3. Task B - ASCII, UTF-8 and Hex Evidence Strings

Digital evidence is often stored as bytes. A byte can represent an ASCII character, part of a Unicode character, a number, or binary data. In this task, you will convert between text and hexadecimal representations.

3.1 Create Text Evidence and View as Hex

```
echo -n "Hello" > evidence_text.txt
cat evidence_text.txt
xxd evidence_text.txt
xxd -p evidence_text.txt
```

3.2 Convert Hex to ASCII

```
# Hex escape method using printf
printf "Hello
"

# Convert plain hex back into bytes using xxd
printf "48656c6c6f" | xxd -r -p
printf "
"
```

3.3 Python Conversion

```
python3 - <<'PY'
import binascii
text = b'Hello'
print('ASCII bytes:', text)
print('ASCII to hex:', binascii.hexlify(text).decode())
print('Hex to ASCII:', binascii.unhexlify('48656c6c6f').decode())
print('ord(H):', ord('H'))
print('chr(72):', chr(72))
PY
```

3.4 Student Exercise

- Create a file named yourfirstname_lab1a.txt containing the text: Digital Forensics Begins With Bytes.
- Use xxd to display the file content in hexadecimal.
- Extract the plain hex using xxd -p.
- Convert the plain hex back to readable text.
- Explain why 0A may appear at the end of output when echo is used without -n.

Forensic explanation requirement

In your report, explain why an examiner should not assume every hexadecimal value is automatically readable text. Some bytes may represent binary structures, images, compressed data, or encrypted content.

4. Task C - Epoch Time and Forensic Timestamps

Timestamps are central to forensic timelines. Incorrect conversion can place a suspect action at the wrong time. This task introduces Unix epoch time and compares it with other epoch systems mentioned in the course material.

4.1 Unix Epoch Conversion

```
# Current Unix timestamp
date +%s

# Convert a known Unix timestamp to readable UTC time
date -u -d @1643643491

# Convert a known Unix timestamp using Python
python3 - <<'PY'
from datetime import datetime, timezone
print(datetime.fromtimestamp(1643643491, tz=timezone.utc))
PY
```

4.2 Compare Local Time and UTC

```
date
date -u
python3 - <<'PY'
from datetime import datetime, timezone
print('Local:', datetime.now())
print('UTC :', datetime.now(timezone.utc))
PY
```

4.3 Mac Absolute Time / Cocoa Core Date Offset

Mac Absolute Time counts seconds since 2001-01-01 00:00:00 UTC. Unix time counts seconds since 1970-01-01 00:00:00 UTC. The difference between the two epochs is 978307200 seconds.

```
python3 - <<'PY'
from datetime import datetime, timezone
unix_epoch = datetime(1970, 1, 1, tzinfo=timezone.utc)
mac_epoch = datetime(2001, 1, 1, tzinfo=timezone.utc)
print(int((mac_epoch - unix_epoch).total_seconds()))

# Convert a sample Mac Absolute Time value to UTC by adding the offset
mac_value = 725846400
unix_value = mac_value + 978307200
print(datetime.fromtimestamp(unix_value, tz=timezone.utc))
PY
```

4.4 Timestamp Worksheet

Timestamp Value	Format	Conversion Command	Readable UTC Result	Screenshot Ref.
1643643491	Unix seconds			
1675008832	Unix seconds			
725846400	Mac Absolute Time seconds			
Current time	Unix seconds			

Important timeline warning

Always state whether your output is UTC or local time. In formal forensic work, timeline confusion caused by timezone mistakes can seriously weaken the reliability of findings.

5. Task D - Hashing and the Newline Problem

Hashing is used to verify that evidence has not changed. Small changes, including one extra newline character, produce a different hash.

5.1 Create Two Similar Files

```
echo "Hello" > hello_with_newline.txt
echo -n "Hello" > hello_without_newline.txt

xxd hello_with_newline.txt
xxd hello_without_newline.txt

md5sum hello_with_newline.txt hello_without_newline.txt
```

```
sha256sum hello_with_newline.txt hello_without_newline.txt
```

5.2 Explain the Difference

- File 1 contains the characters H e l l o plus a newline byte.
- File 2 contains only H e l l o without the newline byte.
- The visible text may look the same, but the underlying bytes are different.
- Because cryptographic hashes are byte-sensitive, the MD5 and SHA-256 values will be different.

5.3 Student Exercise

11. Create two files containing your first name: one using echo and one using echo -n.
12. Use xxd to compare the byte-level difference.
13. Calculate MD5 and SHA-256 for both files.
14. Explain why the hash values differ.
15. State which command you would use if you needed to hash exactly the visible characters only.

6. Task E - Endianness and Data Stored in Memory

Endianness describes the order in which bytes are stored in memory. Digital forensics analysts must understand this when interpreting raw memory, file structures, disk metadata and binary fields.

6.1 Check Byte Order

```
python3 - <<'PY'
import sys
print('System byte order:', sys.byteorder)
PY
```

6.2 Interpret the Same Bytes in Two Ways

```
python3 - <<'PY'
raw = bytes.fromhex('78563412')
print('Raw bytes:', raw.hex())
print('Interpreted as little-endian:', int.from_bytes(raw, 'little'))
print('Interpreted as big-endian    :', int.from_bytes(raw, 'big'))
PY
```

6.3 Student Exercise

Raw Hex Bytes	Little-Endian Decimal	Big-Endian Decimal	Command Used	Explanation
01000000				
FF000000				
00000100				
78563412				

Forensic explanation requirement

In your own words, explain how the same four bytes can produce different decimal values depending on byte order. Relate this to interpreting disk metadata or memory evidence.

7. Task F - Mini Forensic Interpretation Case

You have recovered the following values from a training evidence source. Decode and interpret them using the techniques practiced above.

Evidence Item	Value	Student Interpretation	Command/Screenshot Ref.
EVID-001	48656c6c6f204943444641		
EVID-002	01001000 01101001		
EVID-003	1675008832		
EVID-004	725846400 (Mac Absolute Time)		
EVID-005	78563412 as little-endian and		

Evidence Item	Value	Student Interpretation	Command/Screenshot Ref.
	big-endian		

7.1 Required Case Narrative

- State what each evidence item represents after decoding.
- For timestamp values, state the readable UTC time and the method used.
- For byte-order values, state both interpretations and why they differ.
- Explain which item would be useful for timeline analysis and which item would be useful for text/content interpretation.

8. Submission Requirements

Required output file name

Submit your report as: CIP-B101_Lab1A_FirstName_LastName.pdf

8.1 Your Report Must Contain

- Cover page with course code, lab title, full name, student ID, date and platform used.
- Completed conversion worksheet for decimal, binary and hexadecimal tasks.
- Screenshots of shell/Python validation commands.
- ASCII/hex conversion evidence and explanation.
- Timestamp conversion table with UTC result clearly stated.
- Hash comparison table for files with and without newline characters.
- Endianness worksheet and explanation.
- Mini forensic case interpretation narrative.
- Reflection: one paragraph on why number systems and timestamps matter in digital forensics.

8.2 Minimum Screenshot Checklist

No.	Screenshot Evidence	Included?
1	Working folder creation and pwd output	Yes / No
2	Binary/hex/decimal conversion commands	Yes / No
3	xxd output for text evidence	Yes / No
4	Hex-to-ASCII conversion output	Yes / No
5	Unix timestamp conversion output	Yes / No
6	Mac Absolute Time conversion output	Yes / No
7	Hash comparison using md5sum and sha256sum	Yes / No
8	Endianness Python output	Yes / No
9	Mini-case evidence interpretation commands	Yes / No

9. Marking Rubric

Assessment Area	Marks	What the Instructor Should Check
Manual number-system conversions	15	Correct method, weights/reminders shown, accurate final results.
Command-line and Python validation	15	Commands are correct, outputs are captured and interpreted.
ASCII/hex/UTF-8 understanding	15	Student can explain byte-level text representation and newline issue.
Timestamp conversion and timezone awareness	20	UTC/local distinction is clear; Unix and Mac time are converted correctly.
Hashing and evidence integrity explanation	15	Student explains why byte differences change hashes.
Endianness interpretation	10	Little-endian and big-endian differences are correctly explained.
Report quality and screenshots	10	Report is organized, complete and evidence screenshots are clear.
Pass standard		

Assessment Area	Marks	What the Instructor Should Check
A trainee should be considered competent if they can explain the meaning of the outputs, not merely run the commands.		

10. Reference Links from the Course Material

- https://www.gnu.org/software/bash/manual/html_node/Shell-Arithmetic.html
- <https://linuxize.com/post/bash-printf-command/>
- <https://en.wikipedia.org/wiki/Unicode>
- https://en.wikipedia.org/wiki/Coordinated_Universal_Time
- <https://stackoverflow.com/questions/5693360/why-is-an-md5-hash-created-by-python-different-from-one-created-using-echo-and-m>
- <https://www.iar.com/knowledge/learn/programming/migration-techniques-for-different-endianness/>